

Mastering Continuous Q-Learning & SAC

Complete Beginner-Friendly Breakdown: The Continuous Max Problem, Polyak Averaging, Clipped Twin-Q, Maximum Entropy Derivation & Soft Actor-Critic

Instructor: Prof. Sergey Levine (UC Berkeley)

Companion: DEX-ROB Lab, Tianjin University

Frameworks: Achiam (Spinning Up ch19) + Haarnoja & Levine

0. The "Mental Map": Why Does Lecture 8 Exist?

In Lecture 6, we learned how Actor-Critic combines policy optimization with value estimation. But there exists an alternative branch of Reinforcement Learning: **Value-Based Methods (Q-Learning)**.

Classic Q-learning (such as Deep Q-Networks / DQN used by DeepMind for Atari games) learns a state-action function $Q(s, a)$ and picks the optimal action by taking $\arg\max_a Q(s, a)$.

The Fatal Flaw for Robotics: In continuous robotic manipulation, the action is not a discrete choice among 4 buttons; it is a continuous vector of joint velocities, feed rates, and impedance stiffness values. Finding the maximum of a non-linear neural network over an infinite continuous space is computationally impossible in real-time.

Lecture 8 bridges this gap: Sergey Levine explains how to stabilize deep Q-learning using Target Networks and Double Q-learning, how to handle continuous action spaces, and how **Soft Actor-Critic (SAC)** merges Q-learning with Maximum Entropy to create one of the most powerful off-policy continuous control algorithms in modern robotics.

1. Target Networks

Polyak Averaging:
Freezing Moving Targets

2. Overestimation

Clipped Twin-Q:
 $\min(Q_1, Q_2)$ Solution

3. Continuous Max

Intractable max_a:
CEM vs Actor Maximizer

4. Max Entropy RL

Reward + $\alpha H(\pi)$:
Anti-Freezing Exploration

5. The Deadly Triad

Bootstrapping +
Function Approx + Buffer

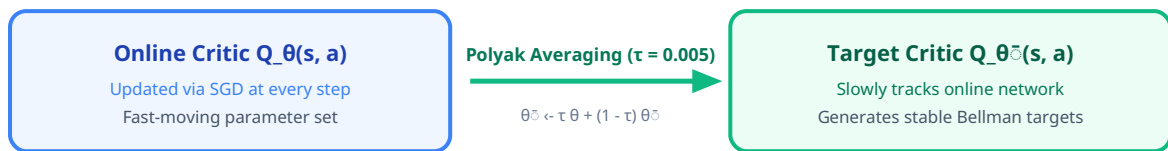
Part 1: Stabilizing Q-Learning: Target Networks & Polyak Averaging

(Slides 1–25, Spoken Transcript 02:20–19:40) In exact tabular Q-learning, the Bellman optimality update is guaranteed to converge. But when we replace the table with a Deep Neural Network $Q_\theta(s, a)$, training frequently explodes. Why?

1.1 The "Chasing Your Own Tail" Problem

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(Q_\theta(s, a) - \left(r + \gamma \max_{a'} Q_\theta(s', a') \right) \right)^2 \right]$$

The network weights θ appear in *both* prediction and target. Every gradient step shifts the target itself! To break this vicious feedback loop, we maintain a secondary set of weights $\bar{\theta}$ called the **Target Network**: $y_t = r_t + \gamma \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a')$.



1.2 Polyak Averaging Half-Life

Instead of hard-copying weights every 10,000 steps, continuous control algorithms smoothly blend target weights after every training step: $\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta}$ (with $\tau = 0.005$). This corresponds to an exponential decay half-life of:

$$t_{\text{half}} = \frac{\ln(2)}{\tau} = \frac{0.693}{0.005} \approx 138 \text{ gradient steps}$$

This prevents abrupt target discontinuities, ensuring smooth impedance gain transitions on physical robotic hardware.

Part 2: Overestimation Bias & Clipped Double Q-Learning

(Slides 26–42, Spoken Transcript 19:50–35:15) A profound mathematical flaw exists in standard Q-learning: **the maximization step systematically overestimates value functions!**

2.1 The Mathematical Origin of Overestimation

By Jensen's inequality and the convexity of the maximum function:

$$\mathbb{E}[\max_a \hat{Q}(s, a)] \geq \max_a \mathbb{E}[\hat{Q}(s, a)] = \max_a Q^{\text{true}}(s, a)$$

If the network randomly overestimates an action (e.g. violent downward slam), the $\max$ greedily selects it, causing value predictions to blow up toward $+\infty$.

Standard Q-Learning (Single Critic)

Takes max over noisy estimates

$$E[\max Q] > \max E[Q]$$

Result: Q-Values explode to $+\infty$!

Clipped Twin-Q (TD3 / SAC)

Trains Two Independent Critics Q_1, Q_2

$$\text{Target: } y = r + \gamma \min(Q_1, Q_2)$$

Result: Bounded, pessimistic value targets

2.2 The Clipped Twin-Q Solution (Fujimoto et al., 2018)

Maintain **two separate Critic networks** (Q_{ϕ_1} and Q_{ϕ_2}) with independent initializations. When computing the Bellman target, always evaluate both and take the **minimum**:

$$y_t = r_t + \gamma \min \left(Q_{\phi_1}(s_{t+1}, \tilde{a}_{t+1}), Q_{\phi_2}(s_{t+1}, \tilde{a}_{t+1}) \right)$$

Taking the minimum injects controlled pessimism, completely eliminating overestimation bias.

Part 3: Continuous Actions & The Intractable Max

(Slides 43–58, Spoken Transcript 35:30–48:00) How do we find $\arg\max_a Q(s, a)$ when action vector $\mathbf{a}_t \in \mathbb{R}^6$ is continuous?

Approach	Mechanism	Pros & Cons for Robotics
Stochastic Optimization (CEM / CMA-ES)	Sample 1,000 random actions, evaluate $Q(s, a)$, fit Gaussian to top 10%, repeat 3 times.	Extremely slow in the inner loop; fails beyond 10–20 action dimensions; unacceptable for real-time 60 Hz execution.
Analytical Q-Functions (NAF)	Constrain $Q(s, a)$ to be strictly quadratic in a : $Q(s, a) = V(s) - \frac{1}{2}(a - \mu)^T P (a - \mu)$.	Maximum is analytically $\mu(s)$, but restricts the Critic to simple parabolic shapes; cannot capture complex contact bifurcations.
Learned Actor Maximizer (DDPG / SAC)	Train a separate neural policy $\pi_\theta(s)$ whose explicit objective is to output the action that maximizes $Q(s, a)$.	The Winning Approach: Computing the maximum requires only a single forward pass through the Actor network $\pi_\theta(s)$!

Part 4: Soft Actor-Critic (SAC) & Maximum Entropy RL

(Slides 59–75 & Spinning Up ch19) Soft Actor-Critic modifies the objective to maximize both **reward** AND **policy entropy** $\mathcal{H}(\pi)$:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right]$$

4.1 The "Anti-Freezing" Phenomenon in Soft Fruit Slicing

Touching a tomato risks incurring our -15 crushing penalty. Under standard RL, the policy collapses into a local minimum where the blade **hovers stationary 1 mm above the skin** to collect safe zero rewards. **Under Maximum Entropy SAC:** A frozen blade has zero entropy ($\mathcal{H} = 0$), which incurs a massive entropy penalty! The robot is compelled to keep vibrating and testing sawing actions, discovering that light lateral sawing punctures the skin cleanly without crushing.

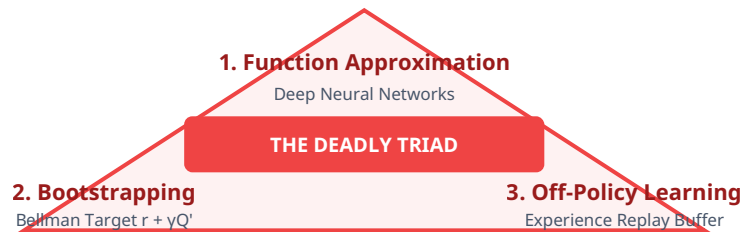
4.2 Automatic Entropy Temperature Tuning (α)

Rather than keeping temperature α fixed, SAC formulates temperature optimization as a dual constrained problem targeting heuristic $\bar{\mathcal{H}} = -\dim(\mathcal{A}) = -6$:

$$\mathcal{L}(\alpha) = \mathbb{E}_{a \sim \pi} \left[-\alpha \log \pi(a|s) + \bar{\mathcal{H}} \right]$$

Part 5: Sutton's "Deadly Triad" in Value-Based RL

(Slides 76–88, Spoken Transcript 1:09:00–1:20:15) Rich Sutton proved that severe instability or divergence in reinforcement learning arises whenever three algorithmic elements are combined simultaneously:



SPINNING UP BUG ALERT: BUFFER VS. CURRENT POLICY ACTIONS

Joshua Achiam highlights a frequent silent bug: In Critic loss, $Q(s, a)$ MUST evaluate historical actions sampled from the replay buffer ($a \sim \mathcal{D}$). In Actor loss, $Q(s, \tilde{a})$ MUST evaluate newly sampled actions from the current policy ($\tilde{a} \sim \pi_{\theta}(s)$). If you evaluate buffer actions in the Actor loss, gradients will not flow from the policy weights, completely freezing policy learning!

Part 6: Paper 1 Implementation Blueprint: SAC Architecture

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def compute_sac_losses(actor, q1, q2, target_q1, target_q2, log_alpha, batch, gamma=0.99):
    states, actions, rewards, next_states, dones = batch
    alpha = log_alpha.exp() # 1. CRITIC LOSS: Bellman Target using Target Twin-Q with torch.no_grad():
    next_actions, next_log_pi = actor.sample(next_states)
    q1_target = target_q1(next_states, next_actions)
    q2_target = target_q2(next_states, next_actions)
    min_next_q = torch.min(q1_target, q2_target)
    next_log_pi = rewards + gamma * (1.0 - dones) * min_next_q
    q1_loss = F.mse_loss(q1(states, actions), next_log_pi)
    q2_loss = F.mse_loss(q2(states, actions), next_log_pi)
    critic_loss = q1_loss + q2_loss # 2. ACTOR LOSS: Reparameterized Gradient
    new_actions, log_pi = actor.sample(states)
    min_q = torch.min(q1(states, new_actions), q2(states, new_actions))
    actor_loss = (alpha.detach() * log_pi - min_q).mean() # 3. TEMPERATURE LOSS: Target entropy heuristic
    target_entropy = -float(actions.shape[-1])
    alpha_loss = -(log_alpha * (log_pi + target_entropy).detach()).mean()
    return critic_loss, actor_loss, alpha_loss
```

Part 7: Interactive Tablet Self-Test Quiz (Test Your Understanding)

Question 1: Why does Clipped Twin-Q use the minimum of two target Q-networks rather than their average?

Answer: Taking the average does not prevent overestimation bias; if both networks have positive noise, their average remains positively biased. Taking the minimum $\min(Q_1, Q_2)$ introduces a mild, controlled underestimation bias that acts as a conservative safety margin, preventing runaway value explosion during contact manipulation.

Question 2: In SAC, why is the target entropy heuristic set to $-\dim(\mathcal{A})$?

Answer: A standard Gaussian distribution $\mathcal{N}(0, I)$ has entropy $\frac{d}{2}(1 + \ln(2\pi))$. Haarnoja and Levine found empirically that setting target entropy $\bar{\mathcal{H}} = -d$ (where $d = \dim(\mathcal{A})$) scales linearly with action space dimensionality, ensuring sufficient exploration across all 6 motor degrees of freedom without overwhelming the task reward.

Question 3: How does Polyak averaging ($\tau = 0.005$) stabilize training on physical robots?

Answer: Polyak averaging smoothly blends target network weights with an exponential half-life of ~ 138 steps ($\bar{\theta} \leftarrow \tau \theta + (1-\tau)\bar{\theta}$). This prevents the sudden value step-discontinuities that occur during periodic hard target copies, eliminating violent torque spikes at robot arm joints.

Part 8: Thesis Defense Master Cheatsheet (Lecture 8 Focus)

Q1: "WHY DOES STANDARD DEEP Q-NETWORKS (DQN) FAIL IN CONTINUOUS ROBOTIC MANIPULATION?"

Answer: "DQN selects actions via $\arg\max_a Q(s, a)$. In discrete games with 4 buttons, this requires 4 forward evaluations. In continuous multi-joint manipulation where the action space is $\mathbb{R}^6$ or $\mathbb{R}^{14}$, finding the continuous global maximum of an arbitrary neural network is an intractable non-convex optimization problem that cannot be solved within our 60 Hz control loop. Algorithms like DDPG and SAC solve this by training a dedicated Actor network to approximate the maximizer directly."

Q2: "WHAT CAUSES OVERESTIMATION BIAS IN Q-LEARNING, AND HOW DOES CLIPPED TWIN-Q ELIMINATE IT?"

Answer: "Overestimation bias stems from Jensen's inequality and noise in neural network function approximation: taking the maximum over noisy estimates yields an expected value strictly greater than the true maximum ($\mathbb{E}[\max X] \geq \max \mathbb{E}[X]$). Clipped Twin-Q trains two independently initialized critics (Q_1, Q_2) and computes target values using their minimum $\min(Q_1, Q_2)$. This introduces a mild, controlled underestimation bias that completely prevents catastrophic value explosion."

Q3: "WHY IS MAXIMUM ENTROPY RL UNIQUELY EFFECTIVE AT PREVENTING THE ROBOT FROM FREEZING?"

Answer: "In delicate tasks like slicing soft fruit, touching the fruit introduces risk of incurring high crushing penalties. Standard RL policies frequently collapse into a trivial local minimum where the blade hovers stationary above the skin to collect 0 reward. Maximum Entropy RL augments the reward with an entropy bonus $\alpha \mathcal{H}(\pi)$. Because a stationary action has zero entropy, freezing is heavily penalized, compelling the policy to continuously explore compliant sawing motions until it masters skin puncture."